

实验作业三 手搓最小 LLM——使用 CPU 训练

（对应教材第 5 章《Transformer 模型》·字符级语言模型·使用 TRAE IDE 完成）

一、实验目的

- 1. 理解语言模型“预测下一个字符”的自监督训练目标，掌握字符级分词、上下文窗口截取与“标签错开一位”（输入取前 T 个、标签取后 T 个）的实现；
- 2. 亲手用 PyTorch 搭建一个约 50 万参数的最小 GPT（词嵌入 + 可学习位置编码 + Pre-Norm Transformer 块 + 权重共享），把教材第 5 章的公式变成能跑的代码；
- 3. 在 CPU 上完成基线训练，绘制 loss 曲线，并用模型自回归生成“诗句”，直观感受温度（temperature）与 top-k 采样对生成质量的影响；
- 4. 通过控制变量实验（学习率、层数、嵌入维度、上下文长度、位置编码）理解各超参数的作用，并与手算参数量相互印证；
- 5. 熟练使用 TRAE IDE 的 AI 对话、代码解释与错误修复能力，规范记录人机协作过程。

二、实验环境

表 1 软件环境清单

项目	要求	说明
操作系统	Windows 10/11 或 macOS	任意能跑 Python 的多核 CPU 电脑
Python	3.9 ~ 3.12	建议 3.10+
PyTorch	≥ 2.0（CPU 版即可）	无需 GPU、无需 CUDA
matplotlib	≥ 3.5	绘制 loss 曲线
硬件	内存 4GB 以上，磁盘空间 < 50MB	训练期间风扇加速属正常现象

环境搭建命令（命令行 / TRAE 终端执行）：

```
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install matplotlib
python min_llm.py --iters 800          # 800 步快速自检，先跑通再全量
```

三、TRAE IDE 使用要点

表 2 TRAE 核心功能与本实验的使用原则

功能	本实验的使用原则
AI Builder / Chat 生成框架	让 AI 按“数据 → 分词器 → 模型 → 训练 → 采样”的顺序逐段生成骨架；每一段必须自己读懂再粘贴，禁止一次让 AI 吐出整个文件后直接运行
代码解释	选中 CausalSelfAttention，让 AI 逐行解释：Q/K/V 一次线性投影、按头拆分（view + transpose）、缩放点积、因果掩码的 -inf 填充
错误修复	遇到 view / transpose / reshape 的形状类报错，先自己在纸上写出各步张量形状，再让 AI 验证分析是否正确
调参与实验设计	表 5 的对照方案由自己设计；可让 AI 检查方案是否满足“单一变量原则”
协作记录	报告的“AI 协作记录”栏须如实记录关键提问、采纳/不采纳及理由（计入报告规范分）

四、原理回顾

4.1 LLM 三要素与“最小版”的对应

表 3 真实 LLM 与本实验最小 LLM 的对应关系

要素	真实 LLM（GPT / Llama 等）	本实验的最小 LLM
数据与分词	数万亿 token；BPE 分词，词表 3 万 ~ 15 万	约 70 首唐诗（约 2000 字）；字符级分词，词表 699
模型	数十至上百层 Transformer，隐藏维数千	2 层 Transformer，128 维，4 头
训练目标	下一 token 预测 + 指令微调 + RLHF	仅下一字符预测（交叉熵）
算力	数千卡 GPU 集群训练数月	笔记本 CPU 约 20 ~ 30 分钟

关键点：架构同源。把隐藏维从 12288 降到 128、层数从 96 降到 2，得到的就是“同一个 Transformer”的缩小版——损失下降、格式学习、温度采样等所有现象依然存在，只是程度缩小。

4.2 基线配置与参数量手算（必做）

表 4 基线配置（必选手算验证）

项	符号	基线值
词表大小	V	运行时打印（约 1.0×10^3 ，随语料而定）
嵌入维度	C (n_embd)	128

注意力头数	H (n_head)	4 (每头 32 维)
Transformer 层数	L (n_layer)	2
上下文长度	T (block_size)	128
MLP 隐层	4C	512, GELU 激活
学习率 / 优化器	lr	1×10^{-3} / AdamW
批大小 $\times$ 训练步数	B $\times$ iters	32×2000

参数量估算公式（偏置与 LayerNorm 占比极小，可忽略）：

总参数 $\approx V \times C$ （词嵌入，与输出层共享只计一次）+ $T \times C$ （位置编码）+ $L \times 12C^2$ （每层注意力 $4C^2$ + MLP $8C^2$ ）

代入基线（运行打印 $V=699$, $C=128$, $T=128$, $L=2$ ）： $699 \times 128 \approx 8.9$ 万 + $128 \times 128 \approx 1.6$ 万 + $2 \times 12 \times 128^2 \approx 39.3$ 万，合计 ≈ 49.9 万，与代码打印的 502,656（差值来自偏置与 LayerNorm）一致，约为 GPT-3（1750 亿参数）的三十五万分之一。请把代入过程写进报告，并与代码打印值核对。

五、实验任务

任务一（基线，必做）：运行 `python min_llm.py` 完成基线训练，记录参数量、最终 loss、训练耗时；验证初始 $\text{loss} \approx \ln V = \ln 699 \approx 6.55$ （体会小方差初始化的作用），保存 loss 曲线（`loss_curve_L2_E128_lr0.001.png`）；用提示词「春」「月」各生成 2 段并保存（`generated_base.txt`）。

任务二（手算验证，必做）：按 4.2 节公式与表 4 手算总参数量，写出代入过程，与代码打印值核对，误差应小于 1%。

任务三（对照实验，必做）：完成表 5 的 6 组对照实验。每组只改一个变量、其余与基线一致；exp1~exp5 建议统一用 `--iters 1000` 缩短时间，记录最终 loss 并给生成质量打星（1~5 星）；exp6 只改采样参数。

表 5 对照实验设计与结果记录表

编号	对照维度	基线值	实验值	最终 loss	生成质量	现象简述
exp1	学习率	1e-3	1e-2 与 1e-4			
exp2	层数 n_layer	2	1 与 4			
exp3	嵌入维度 n_embd	128	64（头数 2）与 256（头数 8）			
exp4	上下文 block_size	128	32			
exp5	位置编码	有	--no_pos 去掉			
exp6	采样温度	1.0	0.5 与 1.5			

			(top_k=20 不变)			
--	--	--	------------------	--	--	--

任务四（进阶选做，二选一）：(a) 自备语料（歌词、新闻、自己的文章等≥1万字，存为 corpus_extra.txt 放同目录，代码会自动追加），对比基线的生成差异，讨论“语料规模与背诵现象”；(b) 给 generate() 增加重复惩罚（如禁止出现过的 3-gram 或频率惩罚），对比前后生成效果。

六、实验步骤

- 步骤 1 用 TRAE Builder 生成实验框架：**新建目录 hw3_min_llm 与文件 min_llm.py；让 AI 按第七节框架逐段生成代码。提示：模型内部所有张量形状为 (B, T, C)——批大小、序列长度、嵌入维度；注意力内部才拆出多头维。
- 步骤 2 手算验证（必做）：**用 4.2 节公式计算基线总参数量，写出代入过程填入报告，并用 sum(p.numel() for p in model.parameters()) 的打印值核对。
- 步骤 3 运行基线并保存结果：**python min_llm.py（CPU 约 20~30 分钟，本机实测约 1.3 it/s ≈ 25 分钟；时间紧可用 --iters 800 先跑通）。保存 loss 曲线与 generated_base.txt，把参数量、最终 loss、耗时填入报告。
- 步骤 4 逐组完成对照实验：**按表 5 逐组实验，每次只改一个变量；每组保存曲线（脚本会自动按配置命名 loss_curve_*.png，如 --n_layer 1 对应 loss_curve_L1_E128_lr0.001.png），并把结果填入表 5。
- 步骤 5 采样实验与生成分析：**固定基线模型，对比 temperature = 0.5 / 1.0 / 1.5 与 top_k = 5 / 20 / 不限制（设为词表大小即可）共 3~4 组生成，每组 3 段；从“确定性 ↔ 多样性”“重复 ↔ 胡言”两个维度分析，给出你认为最优的组合并说明理由。
- 步骤 6（选做） 扩充语料并对比：**按任务四 (a) 追加自备语料，重跑基线，对比生成内容与 loss 曲线的变化。

七、参考代码（完整框架，单文件可直接运行）

以下 min_llm.py 与本指南配套：内置唐诗语料、字符级分词、最小 GPT、训练与采样，无需任何外部数据与网络。可整体复制运行，也可让 TRAE 按段生成后自己拼装（更推荐后者）。

注意：① --no_pos 用于 exp5 去掉位置编码；② 追加语料只需在同目录放 corpus_extra.txt；③ Windows 控制台中文乱码时先执行 chcp 65001，或在 TRAE 内置终端 / PyCharm 中运行。

```
# -*- coding: utf-8 -*-
"""
```

手搓最小 LLM — 使用 CPU 训练（实验作业三参考框架）

对应教材第 5 章《Transformer 模型》

单文件、零依赖外部数据：内置唐诗语料，字符级 GPT，CPU 数分钟~二十分钟训完。

用法示例：

```
python min_llm.py                # 基线：训练 2000 步并生成
python min_llm.py --iters 800    # 快速跑通
python min_llm.py --n_layer 1 --n_embd 64 # 对照实验：改模型
python min_llm.py --no_pos       # 对照实验：去掉位置编码
python min_llm.py --temperature 1.5 # 采样温度
"""

import argparse
import math
import os
import random
import time

import torch
import torch.nn as nn
import torch.nn.functional as F

# ===== 1. 内置语料：精选唐诗（可自行替换/追加） =====
POEMS = [
    "床前明月光，疑是地上霜。举头望明月，低头思故乡。",
    "春眠不觉晓，处处闻啼鸟。夜来风雨声，花落知多少。",
    "白日依山尽，黄河入海流。欲穷千里目，更上一层楼。",
    "锄禾日当午，汗滴禾下土。谁知盘中餐，粒粒皆辛苦。",
    "离离原上草，一岁一枯荣。野火烧不尽，春风吹又生。",
    "远上寒山石径斜，白猿啼不住。停车坐爱枫林晚，霜叶红于二月花。",
    "千山鸟飞绝，万径人踪灭。孤舟蓑笠翁，独钓寒江雪。",
    "鹅，鹅，鹅，曲项向天歌。白毛浮绿水，红掌拨清波。",
    "两个黄鹂鸣翠柳，一行白鹭上青天。窗含西岭千秋雪，门泊东吴万里船。",
    "朝辞白帝彩云间，千里江陵一日还。两岸猿声啼不住，轻舟已过万重山。",
    "故人西辞黄鹤楼，烟花三月下扬州。孤帆远影碧空尽，唯见长江天际流。",
    "李白乘舟将欲行，忽闻岸上踏歌声。桃花潭水深千尺，不及汪伦送我情。",
    "日照香炉生紫烟，遥看瀑布挂前川。飞流直下三千尺，疑是银河落九天。",
    "天门中断楚江开，碧水东流至此回。两岸青山相对出，孤帆一片日边来。",
    "月落乌啼霜满天，江枫渔火对愁眠。姑苏城外寒山寺，夜半钟声到客船。",
    "清明时节雨纷纷，路上行人欲断魂。借问酒家何处有，牧童遥指杏花村。",
    "独在异乡为异客，每逢佳节倍思亲。遥知兄弟登高处，遍插茱萸少一人。",
    "烟笼寒水月笼沙，夜泊秦淮近酒家。商女不知亡国恨，隔江犹唱后庭花。",
    "折戟沉沙铁未销，自将磨洗认前朝。东风不与周郎便，铜雀春深锁二乔。",
    "巴山楚水凄凉地，二十三年弃置身。怀旧空吟闻笛赋，到乡翻似烂柯人。",
    "沉舟侧畔千帆过，病树前头万木春。今日听君歌一曲，暂凭杯酒长精神。",
    "朱雀桥边野草花，乌衣巷口夕阳斜。旧时王谢堂前燕，飞入寻常百姓家。",
    "湖光秋月两相和，潭面无风镜未磨。遥望洞庭山水翠，白银盘里一青螺。",
    "杨柳青青江水平，闻郎江上踏歌声。东边日出西边雨，道是无晴却有晴。",
    "自古逢秋悲寂寥，我言秋日胜春朝。晴空一鹤排云上，便引诗情到碧霄。",
    "前不见古人，后不见来者。念天地之悠悠，独怆然而涕下。",
    "葡萄美酒夜光杯，欲饮琵琶马上催。醉卧沙场君莫笑，古来征战几人回。",
    "黄河远上白云间，一片孤城万仞山。羌笛何须怨杨柳，春风不度玉门关。",
    "秦时明月汉时关，万里长征人未还。但使龙城飞将在，不教胡马度阴山。",
    "月黑雁飞高，单于夜遁逃。欲将轻骑逐，大雪满弓刀。",
    "好雨知时节，当春乃发生。随风潜入夜，润物细无声。",
    "晓看红湿处，花重锦官城。野径云俱黑，江船火独明。",
    "国破山河在，城春草木深。感时花溅泪，恨别鸟惊心。",
    "烽火连三月，家书抵万金。白头搔更短，浑欲不胜簪。",
    "细草微风岸，危樯独夜舟。星垂平野阔，月涌大江流。",
```

"风急天高猿啸哀，渚清沙白鸟飞回。无边落木萧萧下，不尽长江滚滚来。",
 "花近高楼伤客心，万方多难此登临。锦江春色来天地，玉垒浮云变古今。",
 "独怜幽草涧边生，上有黄鹂深树鸣。春潮带雨晚来急，野渡无人舟自横。",
 "天街小雨润如酥，草色遥看近却无。最是一年春好处，绝胜烟柳满皇都。",
 "昔人已乘黄鹤去，此地空余黄鹤楼。黄鹤一去不复返，白云千载空悠悠。",
 "晴川历历汉阳树，芳草萋萋鹦鹉洲。日暮乡关何处是，烟波江上使人愁。",
 "客舍青青柳色新，渭城朝雨浥轻尘。劝君更尽一杯酒，西出阳关无故人。",
 "寒雨连江夜入吴，平明送客楚山孤。洛阳亲友如相问，一片冰心在玉壶。",
 "山光忽西落，池月渐东上。散发乘夕凉，开轩卧闲敞。",
 "荷笠带斜阳，青山独归远。苍苍竹林寺，杳杳钟声晚。",
 "空山不见人，但闻人语响。返景入深林，复照青苔上。",
 "人闲桂花落，夜静春山空。月出惊山鸟，时鸣春涧中。",
 "红豆生南国，春来发几枝。愿君多采撷，此物最相思。",
 "独坐幽篁里，弹琴复长啸。深林人不知，明月来相照。",
 "君自故乡来，应知故乡事。来日绮窗前，寒梅著花未。",
 "山中相送罢，日暮掩柴扉。春草明年绿，王孙归不归。",
 "花间一壶酒，独酌无相亲。举杯邀明月，对影成三人。",
 "小时不识月，呼作白玉盘。又疑瑶台镜，飞在青云端。",
 "长安一片月，万户捣衣声。秋风吹不尽，总是玉关情。",
 "弃我去者，昨日之日不可留。乱我心者，今日之日多烦忧。",
 "抽刀断水水更流，举杯消愁愁更愁。人生在世不称意，明朝散发弄扁舟。",
 "千里黄云白日曛，北风吹雁雪纷纷。莫愁前路无知己，天下谁人不识君。",
 "慈母手中线，游子身上衣。临行密密缝，意恐迟迟归。谁言寸草心，报得三春晖。",
 "山重水复疑无路，柳暗花明又一村。莫笑农家腊酒浑，丰年留客足鸡豚。",
 "纸上得来终觉浅，绝知此事要躬行。古人学问无遗力，少壮功夫老始成。",
 "死去元知万事空，但悲不见九州同。王师北定中原日，家祭无忘告乃翁。",
 "小荷才露尖尖角，早有蜻蜓立上头。泉眼无声惜细流，树阴照水爱晴柔。",
 "接天莲叶无穷碧，映日荷花别样红。毕竟西湖六月中，风光不与四时同。",
 "胜日寻芳泗水滨，无边光景一时新。等闲识得东风面，万紫千红总是春。",
 "半亩方塘一鉴开，天光云影共徘徊。问渠那得清如许，为有源头活水来。",
 "郁孤台下清江水，中间多少行人泪。西北望长安，可怜无数山。",
 "人生自古谁无死，留取丹心照汗青。辛苦遭逢起一经，干戈寥落四周星。",
 "咬定青山不放松，立根原在破岩中。千磨万击还坚劲，任尔东西南北风。",
 "千锤万凿出深山，烈火焚烧若等闲。粉骨碎身浑不怕，要留清白在人间。",
 "浩荡离愁白日斜，吟鞭东指即天涯。落红不是无情物，化作春泥更护花。",
 "九州生气恃风雷，万马齐喑究可哀。我劝天公重抖擞，不拘一格降人才。",
 "力微任重久神疲，再竭衰庸定不支。苟利国家生死以，岂因祸福避趋之。",

]

```

def build_corpus(extra_file=None):
    """拼接语料：若同目录下有 corpus_extra.txt（学生自备），自动追加"""
    text = "\n".join(POEMS)
    if extra_file and os.path.exists(extra_file):
        with open(extra_file, "r", encoding="utf-8") as f:
            text = text + "\n" + f.read()
    return text

# ===== 2. 字符级分词器 =====
class CharTokenizer:
    def __init__(self, text):
        chars = sorted(set(text))
        self.stoi = {ch: i for i, ch in enumerate(chars)}
        self.itos = {i: ch for ch, i in self.stoi.items()}
        self.vocab_size = len(chars)

    def encode(self, s):
        return [self.stoi[c] for c in s if c in self.stoi]
  
```

```

def decode(self, ids):
    return "".join(self.itos[i] for i in ids)

# ===== 3. 最小 GPT 模型 =====
class CausalSelfAttention(nn.Module):
    """带因果掩码的多头自注意力（教材 5.4 节）"""
    def __init__(self, n_embd, n_head, block_size):
        super().__init__()
        assert n_embd % n_head == 0
        self.n_head = n_head
        self.qkv = nn.Linear(n_embd, 3 * n_embd)      # Q/K/V 一次算出
        self.proj = nn.Linear(n_embd, n_embd)         # 输出投影 W_O
        self.register_buffer(
            "mask",
            torch.tril(torch.ones(block_size, block_size)).view(1, 1, block_size,
block_size))

    def forward(self, x):
        B, T, C = x.shape
        q, k, v = self.qkv(x).split(C, dim=2)
        # 拆多头: (B, T, C) -> (B, n_head, T, head_dim)
        q = q.view(B, T, self.n_head, -1).transpose(1, 2)
        k = k.view(B, T, self.n_head, -1).transpose(1, 2)
        v = v.view(B, T, self.n_head, -1).transpose(1, 2)
        att = (q @ k.transpose(-2, -1)) / math.sqrt(k.size(-1)) # 缩放点积
        att = att.masked_fill(self.mask[:, :, :T, :T] == 0, float("-inf")) # 因果掩码
        att = F.softmax(att, dim=-1) # 归一化
        y = att @ v # 加权求和
        y = y.transpose(1, 2).contiguous().view(B, T, C) # 拼接多头
        return self.proj(y)

class Block(nn.Module):
    """Pre-Norm Transformer 块: LN -> 注意力 -> 残差, LN -> FFN -> 残差"""
    def __init__(self, n_embd, n_head, block_size):
        super().__init__()
        self.ln1 = nn.LayerNorm(n_embd)
        self.attn = CausalSelfAttention(n_embd, n_head, block_size)
        self.ln2 = nn.LayerNorm(n_embd)
        self.mlp = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd), nn.GELU(), nn.Linear(4 * n_embd, n_embd))

    def forward(self, x):
        x = x + self.attn(self.ln1(x)) # 残差连接
        x = x + self.mlp(self.ln2(x))
        return x

class MiniGPT(nn.Module):
    def __init__(self, vocab_size, n_embd=128, n_head=4, n_layer=2,
        block_size=128, use_pos=True):
        super().__init__()
        self.block_size = block_size
        self.use_pos = use_pos
        self.tok_emb = nn.Embedding(vocab_size, n_embd) # 词嵌入
        self.pos_emb = nn.Embedding(block_size, n_embd) if use_pos else None # 可学习位置
        self.blocks = nn.ModuleList(
            [Block(n_embd, n_head, block_size) for _ in range(n_layer)])
        self.ln_f = nn.LayerNorm(n_embd)

```

编码

```

        self.head = nn.Linear(n_embd, vocab_size, bias=False)
        self.head.weight = self.tok_emb.weight
        self.apply(self._init_weights)

# 权重共享
# 小方差初始化

def _init_weights(self, m):
    """GPT 标准初始化: 权重  $N(0, 0.02)$ , 偏置清零  $\rightarrow$  初始  $\text{loss} \approx \ln V$ """
    if isinstance(m, nn.Linear):
        nn.init.normal_(m.weight, mean=0.0, std=0.02)
        if m.bias is not None:
            nn.init.zeros_(m.bias)
    elif isinstance(m, nn.Embedding):
        nn.init.normal_(m.weight, mean=0.0, std=0.02)

def forward(self, idx, targets=None):
    B, T = idx.shape
    pos = torch.arange(T, device=idx.device)
    x = self.tok_emb(idx)
    if self.pos_emb is not None:
        x = x + self.pos_emb(pos)
    for blk in self.blocks:
        x = blk(x)
    logits = self.head(self.ln_f(x))
    loss = None
    if targets is not None:
        loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1))
    return logits, loss

@torch.no_grad()
def generate(self, idx, max_new_tokens=200, temperature=1.0, top_k=None):
    """自回归生成: 每步预测下一个字符并拼接回输入 (教材代码 5-6 思想)"""
    self.eval()
    for _ in range(max_new_tokens):
        idx_cond = idx[:, -self.block_size:]
        logits, _ = self(idx_cond)
        logits = logits[:, -1, :] / max(temperature, 1e-8)
        if top_k is not None:
            kth = torch.topk(logits, min(top_k, logits.size(-1)))[0][:, -1:]
            logits[logits < kth] = float("-inf")
        probs = F.softmax(logits, dim=-1)
        next_id = torch.multinomial(probs, 1)
        idx = torch.cat([idx, next_id], dim=1)
    return idx

# ===== 4. 数据批生成: 随机截取 (block_size+1) 的片段 =====
def get_batch(data, block_size, batch_size, device):
    ix = torch.randint(len(data) - block_size - 1, (batch_size,))
    x = torch.stack([data[i:i + block_size] for i in ix])
    y = torch.stack([data[i + 1:i + block_size + 1] for i in ix])
    return x.to(device), y.to(device)

# ===== 5. 主流程 =====
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--iters", type=int, default=2000)
    ap.add_argument("--batch_size", type=int, default=32)
    ap.add_argument("--block_size", type=int, default=128)
    ap.add_argument("--n_embd", type=int, default=128)
    ap.add_argument("--n_head", type=int, default=4)
    ap.add_argument("--n_layer", type=int, default=2)
    ap.add_argument("--lr", type=float, default=1e-3)

```

```

ap.add_argument("--seed", type=int, default=42)
ap.add_argument("--temperature", type=float, default=1.0)
ap.add_argument("--top_k", type=int, default=20)
ap.add_argument("--max_new_tokens", type=int, default=200)
ap.add_argument("--no_pos", action="store_true", help="去掉位置编码（对照实验5）")
ap.add_argument("--extra_corpus", type=str, default="corpus_extra.txt",
                help="自备补充语料文件（可选）")
args = ap.parse_args()

torch.manual_seed(args.seed); random.seed(args.seed)
device = "cpu"
print(f"PyTorch {torch.__version__} | 设备: {device} | 线程数: {torch.get_num_threads()}")

# 数据
text = build_corpus(args.extra_corpus)
tok = CharTokenizer(text)
data = torch.tensor(tok.encode(text), dtype=torch.long)
print(f"语料 {len(text)} 字符 | 词表大小 V = {tok.vocab_size}")

# 模型
model = MiniGPT(tok.vocab_size, args.n_embd, args.n_head, args.n_layer,
                args.block_size, use_pos=not args.no_pos)
n_params = sum(p.numel() for p in model.parameters())
print(f"模型参数量: {n_params:,} ({n_params/1e6:.2f}M) | 配置: "
      f"n_layer={args.n_layer} n_head={args.n_head} n_embd={args.n_embd} "
      f"block={args.block_size} pos={'有' if not args.no_pos else '无'}")

opt = torch.optim.AdamW(model.parameters(), lr=args.lr)

# 训练
losses = []
t0 = time.time()
for step in range(1, args.its + 1):
    xb, yb = get_batch(data, args.block_size, args.batch_size, device)
    _, loss = model(xb, yb)
    opt.zero_grad(); loss.backward(); opt.step()
    losses.append(loss.item())
    if step % 100 == 0 or step == 1:
        speed = step / (time.time() - t0)
        eta = (args.its - step) / speed
        print(f"step {step:5d}/{args.its} | loss {loss.item():.4f} | "
              f"{speed:.2f} it/s | 预计剩余 {eta:.0f}s")
train_time = time.time() - t0
print(f"训练完成, 耗时 {train_time/60:.1f} 分钟 | 最终 loss {losses[-1]:.4f} "
      f"(前100步均值 {sum(losses[:100])/100:.4f})")

# loss 曲线（英文标签避免中文乱码）
try:
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    plt.figure(figsize=(7, 4))
    plt.plot(losses, linewidth=0.8)
    plt.xlabel("Iteration"); plt.ylabel("Cross-Entropy Loss")
    plt.title(f"Training Loss (n_layer={args.n_layer}, n_embd={args.n_embd}, "
              f"lr={args.lr}, pos={'off' if args.no_pos else 'on'})")
    plt.tight_layout()
    tag = "no_pos" if args.no_pos else f"L{args.n_layer}_E{args.n_embd}_lr{args.lr:g}"

```

```
plt.savefig(f"loss_curve_{tag}.png", dpi=150)
print(f"已保存 loss_curve_{tag}.png")
except ImportError:
    print("未安装 matplotlib, 跳过曲线绘制")

# 生成
model.to(device)
prompt = "春"
out = model.generate(torch.tensor([tok.encode(prompt)]), args.max_new_tokens,
                     args.temperature, args.top_k)
sample = tok.decode(out[0].tolist())
print("=" * 50)
print(f"生成示例 (temperature={args.temperature}, top_k={args.top_k}, "
      f"提示词「{prompt}」):")
print(sample)
with open(f"generated_({'no_pos' if args.no_pos else 'base'}).txt",
          "w", encoding="utf-8") as f:
    f.write(sample)

if __name__ == "__main__":
    main()
```

八、实验报告要求

报告以 Word / PDF 提交，命名“学号_姓名_实验作业三.docx”。请直接在配套的《实验作业三_手搓最小 LLM_实验报告样板.docx》中填写，结构为：实验概述 → 基线结果（参数量 / loss 曲线 / 耗时 / 生成示例）→ 参数量手算 → 对照实验（表 5 与曲线）→ 采样分析 → AI 协作记录 → 思考题。

表 6 评分标准（百分制）

项目	分值	评分要点
基线跑通与结果	15	loss 曲线完整且明显下降；参数量、耗时如实记录；有生成示例
参数量手算	15	代入过程完整，与代码打印值误差 < 1%
对照实验	30	6 组齐全、满足单一变量；曲线命名与表 5 记录一一对应
采样分析	15	温度 / top-k 至少 3 组对比，有生成原文与两维度分析
报告规范（含 AI 协作记录）	10	结构完整、图表有编号；如实记录与 TRAE 的协作过程
思考题	10	6 题均有依据作答，允许查资料但须注明来源
进阶任务	5	语料扩充或重复惩罚，有前后对比

九、常见问题

Q1: CPU 训练一组要多久？

基线 2000 步约 20~30 分钟（本机实测 1.3 it/s $\approx$ 25 分钟，视 CPU 而定）；6 组对照用 1000 步每组约 12~15 分钟，全套约 2.5 小时。先 `--iters 800` 跑通全流程，再跑全量；多组实验可挂后台并行写报告。

Q2: loss 不降甚至 NaN？

多为学习率过大（ $1e-2$ 可能发散） $\rightarrow$ 回到 $1e-3$ ；其次检查标签是否与输入错开一位、交叉熵前是否把 logits 展平成 $(-, V)$ 。

Q3: 为什么用字符级分词而不是 BPE？

字符级词表小、零预处理、不需要额外分词库，能把注意力集中到架构本身；其缺点（序列长、单 token 信息少）正是思考题 (4) 要讨论的内容。

Q4: 训练时电脑很卡 / 风扇狂转？

PyTorch 默认占满所有 CPU 核属正常现象。可在文件开头加 `torch.set_num_threads(4)` 限制线程，把算力留给系统；训练时避免开大型软件。

Q5: 生成内容一直循环重复？

温度过低（如 0.5）或 `top_k` 过小会加剧重复——这正是 exp6 要观察的现象。进阶做法见任务四 (b)：重复惩罚。

Q6: 小语料会把诗“背”下来？

会。语料仅几千字时，模型接近“记忆 + 插值”，loss 可降到很低、生成近乎原句——这是小数据下的正常现象，也解释了真实 LLM 为什么必须依靠海量数据才能“泛化”而不是“背诵”（见思考题 5）。

Q7: 每次结果不一样？

固定 `--seed 42` 即可复现。注意：若中途新增 / 修改了 `corpus_extra.txt`，词表与数据都会变化，对照实验各组必须保证语料一致。

十、思考题（在报告样板中作答）

(1) 结合 exp5: 去掉位置编码后，模型还能学到什么？生成的“诗”还成句吗？为什么？（提示：自注意力对输入顺序是“排列不变”的，教材 5.2 节）

(2) 手算：若 `n_embd` 从 128 增至 256、层数从 2 增至 3（V、T 不变），总参数量约为多少？相比基线增长几倍？结合 exp2 / exp3 的实测时间，说明容量与训练成本的权衡。

(3) 温度 $T \rightarrow 0$ 与 $T \rightarrow \infty$ 时, softmax 采样分别退化成什么? 什么任务适合低温、什么场景适合高温? 结合 exp6 的生成结果说明。

(4) 字符级分词的优缺点各是什么? BPE 如何在“词表大小”与“序列长度”之间折中? 为什么真实 LLM 都用 BPE 类分词器?

(5) 本实验是“从零、在数千字语料上训练”; 真实 LLM 是“预训练(海量语料)+ 指令微调 + RLHF”。请从数据量、参数量、训练目标三方面比较两者, 并回答: 把本实验语料扩大 1000 倍, 能得到一个“会聊天”的模型吗? 为什么?

(6) 回顾你与 TRAE 协作实现 CausalSelfAttention 的过程: AI 在哪些环节最有价值? 哪些地方必须自己读懂公式才能改对(如因果掩码的 -inf 填充、多头 reshape 的顺序、权重共享的实现)?